MACHINE LEARNING

WEEK 4 ASSIGNMENT

Github Repo: https://github.com/Aiman-rafiq/Zee-Outsourcing-Internship-ML/tree/main/Week-04-Model-Comparison-and-Tuning

Student Name	Um E Aiman
Emai	uaiman214@gmail.com
Course	Machine Learning
Submitted To	Amna Abdul salam
Submission Date	06 November 2026

1. Introduction

The objective of this assignment was to compare classification models, evaluate them using five-fold cross-validation, tune their hyperparameters, and assess the final selected model on an unseen test set. K-Nearest Neighbors (KNN), Decision Tree, Random Forest, and Gradient Boosting were evaluated using the Student Performance Factors dataset.

2. Dataset and Target Variable

The dataset was obtained from Kaggle and contains 6,607 student records with 20 original columns. It includes information such as study hours, attendance, sleep, parental involvement, and other student-related factors.

Dataset source: https://www.kaggle.com/datasets/lainguyn123/student-performance-factors

2.1 Target Definition

A categorical target named Needs_Support was created from Exam_Score. Students with a score below 65 were assigned class 1 (Needs Support), while students with a score of 65 or above were assigned class 0 (No Support). This threshold is the rule used for this assignment.

Target Class	Definition	Records	Percentage
0 – No Support	Exam_Score ≥ 65	5,155	78.02%
1 – Needs Support	Exam_Score < 65	1,452	21.98%

The target distribution is imbalanced because the No Support class contains a larger number of records. Stratified splitting was therefore used to preserve the class proportions.

3. Data Preparation and Preprocessing

The dataset was checked for missing values and duplicate records. No duplicate rows were found. Missing values were identified in three columns:

Column	Missing Values
Teacher_Quality	78
Parental_Education_Level	90
Distance_from_Home	67

Two features were engineered:

Study_Attendance_Interaction (study hours multiplied by attendance percentage) and Study_Sleep_Ratio (study hours divided by sleep hours). Exam_Score was excluded from the input features because it was used to create the target; including it would reveal the target information to the models.

The resulting input matrix contained 21 features: 8 numerical and 13 categorical. Numerical missing values were handled using median imputation. Categorical missing values were handled using most-frequent imputation, followed by One-Hot Encoding. StandardScaler was applied in the KNN pipeline because KNN is distance-based.

3.1 Training and Test Split

Setting	Value
Training / testing ratio	80% / 20%
Training records	5,285
Test records	1,322
Random state	42
Stratification	Enabled

4. Initial Model Comparison: 5-Fold Cross-Validation

The four classification models were compared using five-fold cross-validation on the training set. Accuracy was used as the scoring metric. The mean score represents average performance across the five folds, while the standard deviation

indicates variation between folds. All comparison and tuning used the training split; the test set was reserved for the final evaluation.

Model	Mean CV Accuracy	Standard Deviation
KNN	83.75%	0.26 percentage points
Decision Tree	87.15%	1.01 percentage points
Random Forest	91.45%	0.56 percentage points
Gradient Boosting	94.29%	0.33 percentage points

	Mean CV Score	Standard Deviation
KNN	0.8375	0.0026
Decision Tree	0.8715	0.0101
Random Forest	0.9145	0.0056
Gradient Boosting	0.9429	0.0033

Figure 1. Initial five-fold cross-validation comparison.

Gradient Boosting achieved the highest initial mean accuracy (94.29%), followed by Random Forest (91.45%). KNN had the lowest mean accuracy (83.75%). Gradient Boosting also had a relatively small standard deviation, indicating consistent scores across the folds.

5. KNN Hyperparameter Tuning

KNN was evaluated with odd values of K from 1 to 15 using five-fold cross-validation. The best value was K = 9, which achieved a mean accuracy of 84.12%. Accuracy increased up to K = 9 and then decreased slightly for larger values, so K = 9 was selected.

K	Mean CV Accuracy
1	79.07%
3	82.06%
5	83.75%
7	84.07%
9	84.12%
11	84.01%
13	83.75%
15	83.58%

	K Value	Mean CV Accuracy
0	1	0.7907
1	3	0.8206
2	5	0.8375
3	7	0.8407
4	9	0.8412
5	11	0.8401
6	13	0.8375
7	15	0.8358

```
Best K: 9
Best CV Accuracy: 0.8412
```

Figure 2. KNN tuning results.

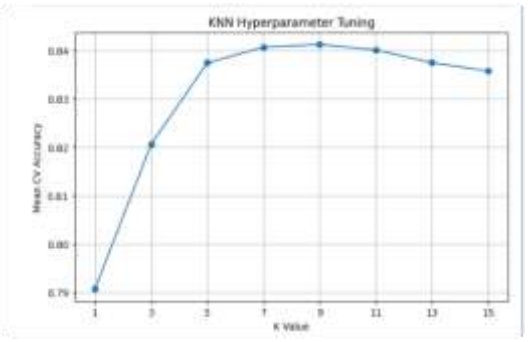


Figure 3. K value versus mean CV accuracy.

6. Decision Tree Hyperparameter Tuning

The Decision Tree model was tested with maximum depth values of 2, 3, 5, 10, and None. The highest mean cross-validation accuracy was 87.44%, obtained at max_depth = 10.

Maximum Depth	Mean CV Accuracy

2	86.43%
3	86.43%
5	87.38%
10	87.44%
None	87.15%

	Max Depth	Mean CV Accuracy
0	2.0	0.8643
1	3.0	0.8643
2	5.0	0.8738
3	10.0	0.8744
4	NaN	0.8715

```
Best Max Depth: 10
Best CV Accuracy: 0.8744
```

Figure 4. Decision Tree tuning results.

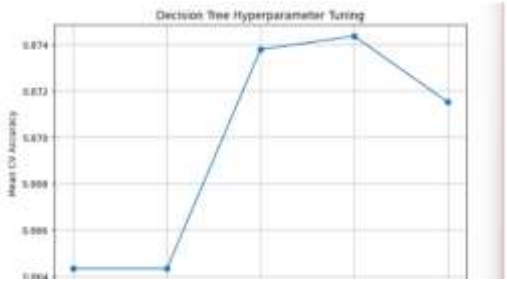


Figure 5. Maximum depth versus mean CV accuracy.

Depth 10 performed slightly better than the other tested values. Allowing the tree to grow without a specified maximum depth did not improve accuracy, so max_depth = 10 was selected.

7. Random Forest Hyperparameter Tuning

GridSearchCV was used to tune Random Forest with n_estimators = [50, 100, 200] and max_depth = [3, 5, None]. This produced 9 parameter combinations. With five folds, the search performed 45 cross-validation fits, followed by one refit of the best estimator on the full training set.

Best Parameter	Selected Value
n_estimators	200
max_depth	None

Best Random Forest CV accuracy: 91.77%

```
Fitting 5 folds for each of 9 candidates, totalling 45 fits
Best Parameters:
{'model__max_depth': None, 'model__n_estimators': 200}

Best Cross-Validation Accuracy:
0.9177
```

Figure 6. Random Forest GridSearchCV output.

The tuned Random Forest improved from the initial 91.45% mean CV accuracy to 91.77%. The best configuration used 200 trees with no specified maximum depth.

8. Gradient Boosting Hyperparameter Tuning

For the extra task, Gradient Boosting was tuned using GridSearchCV. The tested values were n_estimators = [50, 100, 200] and learning_rate = [0.01, 0.05, 0.1, 0.2]. The grid contained 12 combinations and required 60 cross-validation fits, followed by one refit of the best estimator.

Best Parameter	Selected Value
n_estimators	200
learning_rate	0.2

Best Gradient Boosting CV accuracy: 95.52%

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits
Best Parameters:
{'model__learning_rate': 0.2, 'model__n_estimators': 200}

Best Cross-Validation Accuracy:
0.9552
```

Figure 7. Gradient Boosting GridSearchCV output.

Gradient Boosting achieved the highest tuned cross-validation accuracy of all four models. Its accuracy increased from 94.29% before tuning to 95.52% after tuning.

9. Overfitting Analysis

Overfitting was examined by comparing each tuned model's training accuracy with its best cross-validation accuracy. The generalization gap was calculated as training score minus cross-validation score. The holdout test set was not used in this comparison.

Model	Training Score	CV Score	Gap
KNN	88.06%	84.12%	3.94%
Decision Tree	98.26%	87.44%	10.82%
Random Forest	100.00%	91.77%	8.23%
Gradient Boosting	99.51%	95.52%	3.99%

	Model	Training Score	CV Score	Gap
0	KNN	0.8806	0.8412	0.0394
1	Decision Tree	0.9826	0.8744	0.1082
2	Random Forest	1.0000	0.9177	0.0823
3	Gradient Boosting	0.9951	0.9552	0.0399

Figure 8. Training score, cross-validation score, and generalization gap.

Decision Tree had the largest gap (10.82%), indicating the strongest overfitting among the four models. Random Forest also showed a notable gap (8.23%) and reached 100% training accuracy. KNN had the smallest gap (3.94%), while Gradient Boosting had a very similar gap (3.99%) and the highest cross-validation accuracy. Gradient Boosting therefore provided the strongest combination of validation performance and a relatively small gap.

10. Final Test Evaluation

Gradient Boosting achieved the highest tuned CV accuracy (95.52%) and was evaluated on the reserved test set of 1,322 records.

Final test accuracy: 95.23%

```
Best Model: Gradient Boosting
Test Accuracy: 0.9523

Confusion Matrix:
[[1016   15]
 [  48  243]]

Classification Report:
              precision    recall  f1-score   support

   No Support       0.95      0.99      0.97      1031
Needs Support       0.94      0.84      0.89       291

     accuracy                           0.95      1322
    macro avg       0.95      0.91      0.93      1322
 weighted avg       0.95      0.95      0.95      1322
```

Figure 9. Final Gradient Boosting test results and classification report.

10.1 Confusion Matrix

Actual Class	Predicted No Support	Predicted Needs Support
No Support	1,016	15
Needs Support	48	243

The model correctly identified 1,016 No Support records and 243 Needs Support records. It produced 15 false positives and 48 false negatives.

10.2 Classification Metrics and Error Interpretation

Metric	No Support	Needs Support
Precision	95%	94%
Recall	99%	84%
F1-score	97%	89%

Recall was prioritized for Needs Support because false negatives are missed cases. The model achieved 84% recall (243 of 291) and 94% precision; 48 cases were missed.

11. Final Model Selection and Conclusion

Tuned Model	Best CV Accuracy
KNN	84.12%
Decision Tree	87.44%
Random Forest	91.77%
Gradient Boosting	95.52%

Gradient Boosting was selected as the final model because it achieved the highest tuned cross-validation accuracy (95.52%) and a strong test accuracy (95.23%). It also had a relatively small generalization gap of 3.99%, compared with the larger gaps observed for Decision Tree and Random Forest. This assignment provided practical experience in model comparison, five-fold cross-validation, hyperparameter tuning, overfitting analysis, and final model evaluation.

Limitation

One limitation is that Needs_Support is an assignment-defined label based on Exam_Score < 65. It is not an independently verified measure of a student's actual need for support. The reported performance therefore describes prediction of this score-based category, not a validated real-world support decision.